函数解析

`__off_t lseek (int __fd, __off_t __offset, int __whence)`

1.参数解析：
__fd：指文件的标识符（如文件句柄，它是标识和管理已打开文件的一个概念，用于操作文件时的引用，通过它，我们可以对文件进行读取、写入、关闭等操作）
__offset：基于_whence基准的偏移量
__whence：基准位置，有三个标准取值SEEK_SET：
SEEK_SET：以文件开头为基准，新偏移量 = __offset（__offset 必须 ≥ 0）。
SEEK_CUR：以当前偏移量为基准，新偏移量 = 当前偏移量 + __offset。
SEEK_END：以文件末尾为基准，新偏移量 = 文件总长度 + __offset（__offset 为正时可超出文件末尾，形成 “文件空洞”）。

2.返回值：
成功：返回调整后的新偏移量（以文件开头为起点，单位：字节）
失败：返回 -1

3.功能：
调整文件描述符的当前读写偏移量。一般用于文件读写时，定位到文件头，便于后续调用 read() 或 write() 时，会从这个新位置开始操作，并自动更新偏移量。
例如：off_t file_offset = lseek(fd, 50, SEEK_SET);
这里的偏移量为50，而SEEK_SET表示从fd文件开头开始，那么新偏移量就为50，那么接下来的read（）或者write（）操作都将从50字节开始。

`ssize_t write (int __fd, const void *__buf, size_t __n)`

1.参数解析：
__fd：指文件的标识符（如文件句柄，它是标识和管理已打开文件的一个概念，用于操作文件时的引用，通过它，我们可以对文件进行读取、写入、关闭等操作）
__buf：指要写入数据的内存地址
__n：指期望写入的字节数

2.返回值：

成功：返回实际写入的字节数（可能小于 __n，如磁盘空间不足）
失败：返回 -1

3.功能：
将内存中的数据传输到文件或设备。一般用于将缓冲池中的数据写入文件的指定磁盘页面中（write_page（））。

```
ssize_t read (int __fd, void *__buf, size_t __nbytes)
```

1.参数解析：
__fd:指文件的标识符（如文件句柄，它是标识和管理已打开文件的一个概念，用于操作文件时的引用，通过它，我们可以对文件进行读取、写入、关闭等操作）
__buf：指向用户空间的内存地址，用于存储读取的数据
__nbytes：最多读取字节数

2.返回值：
成功：返回实际读取的字节数（0 ≤ 返回值 ≤ __nbytes）。若返回 0，表示已到达文件末尾（EOF）或连接关闭（对套接字而言）。若返回正数 n，表示成功读取 n 字节数据到 __buf 中
失败：返回 -1

3.功能：
将数据从文件或者设备读取到缓存区。一般用于读取磁盘文件中指定编号的页面中的部分数据到内存（缓冲池）中（read_page（））

```
int stat (const char *__restrict __file,struct stat *__r
```

1.参数解析：
__restrict __file：指向文件或者目录路径的字符串
__restrict __buf：指向struct stat的指针，用于存储获取的元数据

2.返回值：
成功：返回 0，元数据存入 __buf
失败：返回 -1

3.功能：
通过文件路径获取其元数据（如大小、权限、修改时间等）。一般搭配stat结构体，用来判断是

否是一个常规文件或者是否是一个目录

`int system (const char *__command)`

1.参数解析：
__command：指向包含 shell 命令的字符串（如 "rm -r test"、"mkdir test"）

2.返回值：
命令执行成功：返回命令的退出状态码（通常为 0 表示成功）。
命令执行失败：返回 -1

3.功能：
在程序中执行外部shell命令。一般用于执行文件和目录的命令操作。

`int chdir (const char *__path)`

1.参数解析：
__path：执行目标目录路径的字符串

2.返回值：
成功：返回 0
失败：返回 -1

3.功能：
是c程序内部的系统调研，用来改变当前进程及其子进程的工作目录。一般用于切换目录执行操作。其于shell命令cd的区别在于shell命令cd会直接改变整个shell会话的进程，影响后序所有命令，而chdir（）只会在c程序的进程中改变工作目录，一但程序结束，shell的工作目录并不会发生变化。

`int open (const char *__path, int __oflag, ...)`

1.参数解析：
__path：指向文件路径的字符串
__oflag：位掩码，指定打开文件的模式和选项（必选）。以下为打开模式和常用选项
打开模式（必选，互斥）

O_RDONLY：只读模式。
O_WRONLY：只写模式。
O_RDWR：读写模式。

常用选项（可选，可组合）
O_CREAT：若文件不存在则创建（需提供第三个参数指定权限）。
O_EXCL：与 O_CREAT 联用，若文件已存在则失败（避免竞态条件）。
O_TRUNC：若文件存在且为可写模式，将其长度截断为 0（清空内容）。
O_APPEND：写入时追加到文件末尾（自动更新文件指针到末尾）。
O_NONBLOCK：非阻塞模式（对设备或管道有效，读写立即返回而非等待）。
O_SYNC：同步写入（数据和元数据同步到磁盘，确保持久性）。
O_TMPFILE：用于创建一个匿名临时文件。这个文件没有文件名，也不会出现在文件系统目录中，但可以像普通文件一样读写，并且在关闭后会自动删除。

...：可变参数，仅在创建文件（O_CREAT 或 O_TMPFILE）时需要，用于指定文件权限（如 0644），以下是文件权限
所有基础权限宏的命名格式为：
S_<操作><用户类别>
操作：R（读）、W（写）、X（执行）。
用户类别：USR（所有者）、GRP（所属组）、OTH（其他用户）。
按 “用户类别” 分类如下：

所有者（User）权限宏
对应 “文件所有者” 的权限，与 S_IRUSR、S_IWUSR 同组：
S_IRUSR：User Read（所有者可读），对应权限位 r，数值 4。
S_IWUSR：User Write（所有者可写），对应权限位 w，数值 2。
S_IXUSR：User Execute（所有者可执行），对应权限位 x，数值 1。
组合示例：
S_IRUSR | S_IWUSR | S_IXUSR → 所有者有读、写、执行权限（rwx），数值 4+2+1=7（八进制 0700 的一部分）。

所属组（Group）权限宏
对应 “文件所属组” 的权限（组内用户的权限）：
S_IRGRP：Group Read（组可读），对应权限位 r，数值 4。
S_IWGRP：Group Write（组可写），对应权限位 w，数值 2。
S_IXGRP：Group Execute（组可执行），对应权限位 x，数值 1。
组合示例：

S_IRGRP | S_IWGRP → 组内用户有读、写权限（rw-），数值 4+2=6（八进制 0060 的一部分）。

其他用户（Others）权限宏
对应 “既不是所有者也不在所属组” 的用户的权限：
S_IROTH：Others Read（其他用户可读），对应权限位 r，数值 4。
S_IWOTH：Others Write（其他用户可写），对应权限位 w，数值 2。
S_IXOTH：Others Execute（其他用户可执行），对应权限位 x，数值 1。
组合示例：
S_IROTH → 其他用户仅有读权限（r--），数值 4（八进制 0004 的一部分）。

2.返回值：
成功：返回文件描述符（如 3、4 等，0、1、2 为标准输入 / 输出 / 错误）
失败：返回 -1

3.功能：
常用于打开已存在的文件或者创建新的文件。

`int unlink (const char *__name)`

1.参数解析：
__name:指要删除的文件或者符号链接的路径字符串

2.返回值：
成功：返回0
失败：返回-1

3.功能：
常用于已关闭文件的删除（未关闭，文件内容不会被删除）

`int close (int __fd)`

1.参数解析：
__fd:要关闭的文件标识符

2.返回值：
成功：返回0

失败：返回-1

3.功能：
常用于文件操作完成后关闭

`void reset_memory()`

1.功能：
一般用于重置页面数据，清除旧数据

`void *memcpy (void *__restrict __dest, const void *__res`

1.参数解析：
__dest：指向目标内存区域的指针（复制的目的地）
__src：指向源内存区域的指针（复制的来源）
__n：要复制的字节数（size_t 类型，无符号整数）

2.功能：
一般用于内存块的数据复制

`typename std::remove_reference<_Tp>::type&& move(_Tp&& _`

1.参数解析：
_Tp&& __t：万能引用，既能接收左值，也能接收右值。
左值：_Tp& && → _Tp&（左值引用）
右值：_Tp && → _Tp&&（右值引用）

2.功能：

常用于移动语义，将对象转换为右值引用。
例如：`std::vector<int> b = std::move(a);`
此时a的资源直接转移给b，且a为空。

`void *memset (void *__s, int __c, size_t __n);`

1.参数解析：
__s：指向目标内存区域的指针（需为可写内存）。
__c：要设置的值（通常是 0 或其他单字节值）。
注意：虽然参数类型是 int，但实际只会使用其低 8 位（即 unsigned char）。
__n：要设置的字节数（size_t 类型，无符号整数）。

2.功能：
一般用来按字节设置内存。例如：memset（buffer，0，10）；表示将buffer的10个字节设置为0

`std::scoped_lock lock{latch_}`

关键组件
std::scoped_lock

特性：
C++17 引入的通用锁管理类，支持多锁获取（防止死锁）。
构造时锁定所有给定的互斥锁，析构时释放。

`latch_` 通常是 `std::mutex` 或 `std::shared_mutex` 类型的成员变量，用于保护共享资源。

1. 功能概述
   作用：在当前作用域内获取 `latch_` 互斥锁，确保同一时间只有一个线程可以执行锁保护的代码块。
   自动释放：std::scoped_lock 是 RAII（资源获取即初始化）类型，锁的生命周期与 lock 对象绑定。当 lock 离开作用域时（如函数返回或遇到 `}`），锁会自动释放。

例如：

```
void addRecord(RmRecord record) {
    std::scoped_lock lock{latch_};  // 获取锁
    // 临界区：只有一个线程可执行此代码块
    records_.push_back(std::move(record));
}  // 锁在此自动释放
```

2.如何判断是否需要锁

检查函数是否访问共享资源：如果只使用局部变量，通常不需要锁。
检查操作是否线程安全：如果操作涉及共享资源的读写，且不是原子（不可被中断的操作）的，需要锁。

`std::unique_ptr`

1.功能：
是 C++ 标准库提供的一种智能指针，专门用于独占式管理动态分配的内存资源。
它的核心作用是：自动释放所指向的对象，避免内存泄漏，同时确保同一时间只有一个指针拥有该对象的所有权。
用一个例子理解：
假设你需要创建一个 RmRecord 对象（比如数据库中的一条记录），传统方式可能用裸指针：

// 裸指针方式（需要手动管理内存）
`RmRecord* record = new RmRecord(100);  // 动态分配内存`

// ... 使用 record ...
`delete record;  // 必须手动释放，否则内存泄漏`
如果忘记 delete，或者在释放前发生异常，就会导致内存泄漏。

而用 std::unique_ptr 则可以避免这个问题：

// unique_ptr 方式（自动管理内存）

```
std::unique_ptr<RmRecord> ret(new RmRecord(100));  // 独占所有
// ... 使用 ret（通过 ret-> 访问成员） ...
// 无需手动 delete! 当 ret 离开作用域时（比如函数结束），会自动释放内存
```

2.特性
-独占所有权：一个 unique_ptr 实例独占它所指向的对象，不能被复制（避免多个指针同时管理同一个对象），只能通过 std::move 转移所有权；
-自动释放资源：当 unique_ptr 离开作用域（比如函数返回、代码块结束）时，会自动调用 delete 释放所指向的对象，无需手动干预，从根源上避免内存泄漏；

`std::make_unique`

1.功能：
自动完成动态内存分配和 unique_ptr 的初始化，避免手动使用 new 带来的潜在风险

// 手动方式（C++11）
```
std::unique_ptr<RmRecord> ptr(new RmRecord(100));  // 先 new，再传递给 unique_ptr
```

// 使用 make_unique（C++14+）
```
std::unique_ptr<RmRecord>  ptr = std::make_unique<RmRecord>(100);  // 自动推导类型，无需重复写 RmReco
```

`std::ofstream ofs(DB_META_NAME)`

1.参数解析：
DB_META_NAME是一个文件，专门用于存储数据库的元信息。

2.功能：
ofstream会在当前目录创建(如果没有此文件先创建)和打开一个名为DB_META_NAME的文件，并通过`<<`将信息写入DB_META_NAME这个文件中去。
比如：`ofs << *new_db`将我们创建的一个新的数据库写入到DB_META_NAME文件中去。

`std::ifstream ofs(DB_META_NAME)`

1.参数解析：
DB_META_NAME是一个文件，专门用于存储数据库的元信息。

2.功能：
ofstream会在当前目录创建(如果没有此文件先创建)和打开一个名为DB_META_NAME的文件，并通过`>>`将里面的文件写出到某一个文件信息中。
比如：`ofs >> db_`就是将DB_META_NAME里面的信息写出到名为db_的数据库中去。

`std::fstream outfile`

1.参数解析：
outfile是一个文件流对象，支持文件的读写操作（即可以用它来打开文件进行输入和输出操

作）。

2.功能：
主要用来处理文件的读写操作。
比如下面这一段语句：

```
outfile.open("output.txt", std::ios::out | std::ios::app);
outfile << "failure\n";
outfile.close();
```

最终实现的效果是向output.txt文件中写入了"failure",并进行了换行。我们来分析一下每句话的功能和作用。

outfile.open(), 第一个参数是创建或打开名为"output.txt"文件。
std::ios::out：表示以输出（写入）模式打开文件，允许向文件中写入数据。
std::ios::app：表示 append（追加）模式，即所有写入操作都将从文件的末尾开始，不会覆盖已有的内容。

outfile << "failure\n" 表示将"failure\n"写入到outfile文件中去。

最终再关闭这个文件。

报错函数解析：

`throw UnixError()`

1.功能：
系统调用相关的错误，比如：close，open，unlink函数返回值异常，系统调用抛出错误。

`throw FileExistsError(path)`

1.参数解析：
string path：想要后序错误信息输出的字符串

2.功能：
文件已经存在报的错误，比如文件已经存在了继续创建文件。当我们需要文件一定不存在的时候，需要加上这段操作。

`throw FileNotClosedError（path）`

1.参数解析：
string path：想要后序错误信息输出的字符串

2.功能：
文件没有被关闭报的错误，比如文件已经是打开的了，我们任然需要打开这个文件。当我们需要文件一定是关闭的时候，需要加上这段操作。

`throw FileNotFoundError（path）`

1.参数解析：
string path：想要后序错误信息输出的字符串

2.功能：
文件没有被找到的错误，比如我们需要删除这个文件，但是不存在这个文件。当我们需要文件一定是存在的时候，需要加上这段操作。

`throw FileNotOpenError（path）`

1.参数解析：
string path：想要后序错误信息输出的字符串

2.功能：
文件没有被打开报的错误，比如关闭文件的时候文件已经关闭了。当我们需要文件一定为打开的时候，需要加上这段操作。

`throw DatabaseExistsError(db_name)`

1.参数解析：
string db_name：表示数据库的名称，用于在错误信息中指出具体哪个数据库已经存在。

2.功能：
当程序尝试创建一个已经存在的数据库时，需要抛出 DatabaseExistsError 异常，用来提醒用户当前数据库已经被创建了。

```
throw DatabaseNotFoundError(db_name)
```

1.参数解析：
string db_name：该参数将在错误信息中被引用，用于明确指出哪个数据库未找到或不存在。

2.功能：
当程序尝试访问一个不存在的数据库时，抛出 DatabaseNotFoundError 异常。再打开或者删除数据库时会使用到。

事务

1.含义：
事务（Transaction） 就是一组 “要么全做完，要么全不做” 的操作。事务的作用就是保证这组操作的 “完整性”，避免出现中间出错导致的数据混乱。

2.特性：

原子性（Atomicity）：“要么全成，要么全废”

原子性是事务的核心，意思是这组操作是 “不可分割” 的 —— 要么两个步骤都成功执行，要么两个都不执行，绝对不会出现 “只做了一半” 的情况。
比如转账时：
如果步骤 1 成功（A 扣了 100），但步骤 2 失败（B 没加 100），事务会 “回滚”（撤销步骤 1 的操作，A 的钱恢复原样），避免 A 的钱白白消失。
如果步骤 1 就失败（比如 A 余额不足），那步骤 2 根本不会执行，B 的钱不变。

一致性（Consistency）：“操作前后，数据规则不变”

一致性指的是事务执行后，数据库的 “业务规则” 依然成立。比如转账的核心规则是 “总金额不变”（A 的钱 + B 的钱 = 总钱数）。
转账前：A 有 500，B 有 300，总金额 800。
转账后：A 有 400，B 有 400，总金额还是 800（规则成立）。
如果事务执行中出问题（比如 A 扣了 100 但 B 没加），总金额会变成 700（规则被破坏），这时候事务会回滚，保证一致性。

隔离性（Isolation）：“多个事务同时执行，互不干扰”

如果多个事务同时操作数据库（比如你给 B 转账的同时，C 也给 B 转账），隔离性保证它们之间不会互相影响，就像 “各干各的” 一样。
比如：
你给 B 转 100（事务 1），C 给 B 转 200（事务 2）。
隔离性确保：要么事务 1 先执行完（B 先加 100，再加 200，最终 + 300），要么事务 2 先执行完（B 先加 200，再加 100，最终 + 300），不会出现 “B 只加了 200” 或者 “B 加了 400” 的错误。

持久性（Durability）：“一旦完成，结果就永久保存”

事务成功执行后，它对数据的修改会 “永久保存”，就算数据库崩溃、断电，重启后结果也不会丢失。
比如转账成功后：
即使银行系统突然断电，重启后 A 的余额还是 400，B 的余额还是 400，不会回到转账前的状态。

目录，文件，页，记录之间的关系。

目录（文件夹）：
数据库的元数据集合，记录数据库中所有对象（表、索引等）的原信息。我们目前遇到的是一个数据库对应一个目录的存储设计。

文件：
每个数据库对象（表、索引）通常对应一个或多个文件。目前我们代码所体现的关系是一个表就对应一个独立文件，表的实际记录数据存储在以表名命名的文件中。一个文件包含多个页，是页的容器。

页：
是数据库I/O的基本单位（磁盘读写必须按照页面划分）。一页包含多条记录；可被加载到内存帧中去。

记录：
数据库中存储的最小逻辑数据单元，由多个字段组成。一条记录就是表的一行数据。如下就是一条记录：

```
+--------+------------+--------+
| id(4B) | name(10B)  | age(4B)|
+--------+------------+--------+
offset:   0           4        14
```

相互关系（包含关系如下）：
记录→页面→文件→目录

记录（Record）→ 页面（Page）

记录是逻辑数据单元，必须 “打包” 到页面中才能存储。一个页面可包含多条记录（数量取决于页面大小和记录大小），例如：8KB 的页面可存储 100 条 80 字节的记录。
页面内部有固定格式（如页头、记录区），页头记录页面元信息（如记录数、空闲空间地址）。

页面（Page）→ 文件（File）

页面是文件的最小物理单位，一个文件由多个页面组成。例如：一张表的所有记录被分散存储在其对应文件的多个页面中。

文件（File）→ 目录（Directory）

目录是数据库的 “索引”，记录每个文件对应的数据库对象（如表名→文件路径）、文件的页面分布、结构定义（如记录的字段类型、长度）等。例如：查询表user时，通过目录（数据库)找到user对应的文件路径，再定位到存储数据的页面。

一句话: 数据按 “记录打包到页面，页面组成文件，文件由目录管理” 的结构存储在磁盘。